

```

"Combatant.h"
-----
#pragma once

#include <Urho3D/Scene/Component.h>
#include <Urho3D/Graphics/AnimationController.h>
#include <Urho3D/Scene/Node.h>
#include <Urho3D/Core/StringUtils.h>

using namespace Urho3D;

enum SIDE { SIDE_HERO = 0, SIDE_VILLAIN = 1 };

class Combatant : public Component
{
    URHO3D_OBJECT(Combatant, Component);

public:
    Combatant(Context* context)
        : Component(context)
        , node_(nullptr)
        , animController_(nullptr)
        , isFrozen_(false)
        , freezeTimer_(0.0f)
        , side_(0)
        , maxHealth_(100)
        , health_(100)
    {}

    virtual void Initialize(Node* node, const ea::string& prefabPath) = 0;
    virtual void Update(float timeStep) = 0;
    virtual bool CheckCollision(Combatant* other) = 0;
    virtual void HandleHit(Combatant* attacker) = 0;

    // Make SetFrozen VIRTUAL so Zombie can override it
    virtual void SetFrozen(bool freeze, bool lockAnimation = true) = 0;

    virtual void Reset() = 0;

    // Common utility accessors
    virtual float GetForwardOffset() const { return 0.0f; } // Added for lunge tracking

    Node* GetNode() const { return node_; }
    Vector3 GetPosition() const { return node_ ? node_->GetWorldPosition() : Vector3::ZERO; }
    void SetPosition(const Vector3& pos) { if (node_) node_->SetWorldPosition(pos); }

    bool IsFrozen() const { return isFrozen_; }

protected:
    Node* node_;
    AnimationController* animController_;
    bool isFrozen_;
    float freezeTimer_;
    int side_;
    int maxHealth_;
    int health_;
};

"CombatSystem.cpp"
-----
#include "CombatSystem.h"
#include <Urho3D/IO/Log.h>
#include <cmath>

CombatSystem::CombatSystem(Context* context)
    : Component(context)
    , fighter_(nullptr)
    , zombie_(nullptr)
    , debugText_(nullptr)
    , arenaHitFrozen_(false)

```

```

{
}

void CombatSystem::Initialize(Fighter* fighter, Zombie* zombie, Text* uiText)
{
    fighter_ = fighter;
    zombie_ = zombie;
    debugText_ = uiText;
    arenaHitFrozen_ = false;
}

void CombatSystem::UpdateCombat(float timeStep)
{
    if (!fighter_ || !zombie_) return;

    // DO NOT call fighter_->Update() or zombie_->Update() here!
    // They are LogicComponents – the engine calls Update() automatically.

    // 1. Run collision detection to maintain safe distance
    if (!arenaHitFrozen_ && !zombie_->IsNeckBiting())
    {
        zombie_->CheckCollision(fighter_);
    }

    // 2. Check if weapon strike connects during the active swing window
    if (fighter_->IsMidSwing() && !zombie_->IsFrozen() && zombie_->IsContactFrozen() &&
    !arenaHitFrozen_)
    {
        float deltaX = std::abs(fighter_->GetPosition().x_ - zombie_->GetPosition().x_);

        if (deltaX < 1.4f)
        {
            URHO3D_LOGINFOF("[COMBAT-HIT] Weapon hit! DeltaX gap: %.2f. SwingTimer active.",
            deltaX);
            arenaHitFrozen_ = true;

            zombie_->HandleHit(fighter_);

            if (debugText_)
            {
                debugText_->SetText("CHOP DETECTED! PRESS SPACEBAR TO SPAWN NEXT RUNNER.");
                debugText_->SetColor(Color::RED);
            }
            return;
        }
    }

    // 3. Bite Sequence with Delay
    if (!arenaHitFrozen_ && zombie_->IsContactFrozen() && !zombie_->IsNeckBiting() && !zombie_-
    >IsFrozen() && !fighter_->IsMidSwing())
    {
        static float biteDelayTimer = 0.0f;
        biteDelayTimer += timeStep;

        if (biteDelayTimer > 0.75f)
        {
            zombie_->StartNeckBite(fighter_->GetNode());
            biteDelayTimer = 0.0f;

            if (debugText_)
            {
                debugText_->SetText("NECK BITE! YOU DIED!");
                debugText_->SetColor(Color::YELLOW);
            }
        }
    }
}

void CombatSystem::HandleManualAttack()

```

```

{
    if (!fighter_ || !zombie_) return;

    // Reset ONLY when zombie is dead (frozen after hit) or player is being bitten
    if (zombie_->IsFrozen() || zombie_->IsNeckBiting())
    {
        URHO3D_LOGINFO("[SYSTEM-INPUT] SPACEBAR: Resetting arena stage...");

        arenaHitFrozen_ = false;
        fighter_->Reset();
        zombie_->Reset();

        if (debugText_)
        {
            debugText_->SetText("STAGE ACTIVE: RUNNING...");
            debugText_->SetColor(Color::GREEN);
        }
        return;
    }

    // Normal attack – works even when zombie is contact-frozen
    URHO3D_LOGINFO("[SYSTEM-INPUT] SPACEBAR: Executing fighter axe swing command.");
    fighter_->StartAttack();
}

void CombatSystem::ResetSimulation()
{
    arenaHitFrozen_ = false;
    if (fighter_) fighter_->Reset();
    if (zombie_) zombie_->Reset();

    if (debugText_)
    {
        debugText_->SetText("STAGE ACTIVE: RUNNING...");
        debugText_->SetColor(Color::GREEN);
    }
}

"CombatSystem.h"
-----
#pragma once

#include <Urho3D/Scene/Component.h>
#include <Urho3D/UI/Text.h>
#include "Fighter.h"
#include "Zombie.h"

using namespace Urho3D;

class CombatSystem : public Component
{
    URHO3D_OBJECT(CombatSystem, Component);

public:
    CombatSystem(Context* context);

    static void RegisterObject(Context* context) { context->RegisterFactory<CombatSystem>(); }

    void Initialize(Fighter* fighter, Zombie* zombie, Text* uiText);
    void UpdateCombat(float timeStep);
    void HandleManualAttack();
    void ResetSimulation();

    bool IsArenaHitFrozen() const { return arenaHitFrozen_; }

private:
    Fighter* fighter_;
    Zombie* zombie_;
    Text* debugText_;

```

```

    bool arenaHitFrozen_;
};
"Fighter.cpp"
-----
#include "Fighter.h"
#include <Urho3D/IO/Log.h>
#include <Urho3D/Core/Context.h>
#include <Urho3D/Resource/ResourceCache.h>
#include <Urho3D/Scene/PrefabResource.h>
#include <Urho3D/Scene/Scene.h>
#include <Urho3D/Graphics/StaticModel.h>
#include <Urho3D/Graphics/Model.h>
#include <Urho3D/Graphics/Skeleton.h>
#include <Urho3D/Graphics/AnimatedModel.h>
#include <Urho3D/Physics/PhysicsWorld.h>
#include <cstring>

Fighter::Fighter(Context* context) :
    LogicComponent(context)
{
    SetUpdateEventMask(USE_FIXEDUPDATE);
}

void Fighter::RegisterObject(Context* context)
{
    context->RegisterFactory<Fighter>();
}

void Fighter::Initialize(Node* node, const ea::string& prefabPath)
{
    node_ = node;

    idleAnimPath_.clear();
    attackAnimPath_ = "test.glb.d/Animations/mixamo.com.ani";

    if (node_)
    {
        arenaPosition_ = node_->GetPosition();
        URHO3D_LOGINFOF("[FIGHTER-INIT] Arena position set: (%.2f, %.2f, %.2f)",
            arenaPosition_.x_, arenaPosition_.y_, arenaPosition_.z_);

        auto* cache = GetSubsystem<ResourceCache>();
        if (cache && !prefabPath.empty())
        {
            auto* prefabFile = cache->GetResource<PrefabResource>(prefabPath);
            if (prefabFile)
            {
                meshNode_ = node_->CreateChild("AxeManMesh");
                prefabFile->InstantiateReference(meshNode_);
                URHO3D_LOGINFOF("[FIGHTER-INIT] Prefab instantiated on child mesh node: %s",
                    prefabPath.c_str());

                meshNode_->SetRotation(Quaternion(0.0f, -90.0f, 0.0f));
                meshNode_->SetScale(5.0f / 9.0f);

                Vector3 pos = meshNode_->GetPosition();
                pos.y_ -= 0.05f;
                meshNode_->SetPosition(pos);

                animController_ = meshNode_->FindComponent<AnimationController>();

                if (animController_) {
                    URHO3D_LOGINFO("[FIGHTER-INIT] AnimationController found on mesh node.");
                    animController_->StopAll();
                } else {
                    URHO3D_LOGWARNING("[FIGHTER-INIT] AnimationController NOT found on mesh
node!");
                }
            }
        }
    }
}

```

```

    animatedModel_ = meshNode_->FindComponent<AnimatedModel>();
    if (animatedModel_)
    {
        Skeleton& skeleton = animatedModel_->GetSkeleton();
        const char* rootNames[] = {
            "mixamorig:Hips", "mixamorig_Hips", "Hips",
            "mixamorig:Root", "mixamorig_Root", "Root",
            "mixamorig:Pelvis", "mixamorig_Pelvis", "Pelvis"
        };
        for (const char* name : rootNames)
        {
            rootBone_ = skeleton.GetBone(name);
            if (rootBone_) break;
        }
        if (rootBone_)
            URH03D_LOGINFOF("[FIGHTER-INIT] Root bone found: %s", rootBone_-
>name_.c_str());
        else
            URH03D_LOGWARNING("[FIGHTER-INIT] Root bone NOT found! Treadmill will not
work.");
    }

    ReparentWeapon(meshNode_);
}

SetupPhysics();
}

Reset();
}

void Fighter::Initialize(Node* node, const ea::string& idleAnimPath, const ea::string&
attackAnimPath)
{
    node_ = node;
    idleAnimPath_ = idleAnimPath;
    attackAnimPath_ = attackAnimPath;

    if (node_)
    {
        arenaPosition_ = node_->GetPosition();
        animController_ = node_->GetOrCreateComponent<AnimationController>();

        if (animController_) {
            URH03D_LOGINFOF("[FIGHTER-INIT] AnimationController initialized successfully.");
        }

        SetupPhysics();
    }

    Reset();
}

void Fighter::SetupPhysics()
{
    if (!node_) return;

    // Create RigidBody if not exists
    rigidBody_ = node_->GetOrCreateComponent<RigidBody>();
    if (rigidBody_)
    {
        rigidBody_->SetMass(80.0f);
        rigidBody_->SetUseGravity(false); // No gravity in this arena
        rigidBody_->SetLinearDamping(10.0f); // High damping to stop quickly
        rigidBody_->SetAngularDamping(10.0f);
        rigidBody_->SetKinematic(false); // Dynamic, responds to forces
        rigidBody_->SetCollisionLayer(3); // Fighter layer
        rigidBody_->SetCollisionMask(0xFFFFFFFF); // Collide with everything
    }
}

```

```

        URH03D_LOGINFO("[FIGHTER-PHYSICS] RigidBody created. Mass=80kg, Gravity=OFF,
Kinematic=OFF");
    }

    // Create CollisionShape if not exists
    collisionShape_ = node_->GetOrCreateComponent<CollisionShape>();
    if (collisionShape_)
    {
        // Capsule: radius=0.3m, height=1.6m (humanoid)
        collisionShape_->SetCapsule(0.3f, 1.6f, Vector3(0.0f, 0.8f, 0.0f));
        URH03D_LOGINFO("[FIGHTER-PHYSICS] Capsule collider created. R=0.3, H=1.6");
    }
}

void Fighter::Reset()
{
    isSwinging_ = false;
    swingTimer_ = 0.0f;
    rootPosInitialized_ = false;

    if (node_)
    {
        node_->SetPosition(arenaPosition_);
        node_->SetRotation(Quaternion::IDENTITY);

        if (rigidBody_)
        {
            rigidBody_->SetPosition(arenaPosition_);
            rigidBody_->SetRotation(Quaternion::IDENTITY);
            rigidBody_->SetLinearVelocity(Vector3::ZERO);
            rigidBody_->SetAngularVelocity(Vector3::ZERO);
            rigidBody_->ResetForces();
            rigidBody_->Activate();
        }
    }

    if (animController_)
    {
        animController_->StopAll();
    }

    URH03D_LOGINFOF("[FIGHTER-RESET] Position reset to arena: (%.2f, %.2f, %.2f)",
        arenaPosition_.x_, arenaPosition_.y_, arenaPosition_.z_);
}

void Fighter::StartAttack()
{
    if (isSwinging_) return;

    if (!animController_) return;

    isSwinging_ = true;
    swingTimer_ = 0.0f;
    rootPosInitialized_ = false;

    animController_->StopAll();

    AnimationParameters params(context_, attackAnimPath_);
    params.looped_ = false;
    animController_->PlayNewExclusive(params);
    animController_->SetTime(attackAnimPath_, 0.0f);
    animController_->SetSpeed(attackAnimPath_, 1.0f);

    URH03D_LOGINFO("[FIGHTER-ATTACK] Attack started. Swing window: 0.20s - 0.60s");
}

// TREADMILL: Cancel root motion by snapping the root bone back to the pivot position
void Fighter::CancelRootMotion()

```

```

{
    if (!rootBone_ || !rootBone_->node_ || !meshNode_ || !node_) return;

    Vector3 rootWorld = rootBone_->node_->GetWorldPosition();
    Vector3 nodeWorld = node_->GetWorldPosition();

    if (!rootPosInitialized_)
    {
        baselineY_ = rootWorld.y_ - nodeWorld.y_;
        rootPosInitialized_ = true;
    }

    Vector3 desired(nodeWorld.x_, nodeWorld.y_ + baselineY_, nodeWorld.z_);
    Vector3 delta = desired - rootWorld;

    if (delta.Length() > 0.001f)
    {
        meshNode_->Translate(delta, TS_WORLD);
    }
}

void Fighter::FixedUpdate(float timeStep)
{
    if (node_)
        URH3D_LOGINFOF("[FIGHTER-POS] x=%.2f", node_->GetPosition().x_);
    // ONLY cancel root motion if an animation is playing that could cause drift
    if (animController_ && animController_->IsPlaying(attackAnimPath_))
        CancelRootMotion();

    if (isSwinging_)
    {
        swingTimer_ += timeStep;

        if (!animController_->IsPlaying(attackAnimPath_))
        {
            isSwinging_ = false;
            swingTimer_ = 0.0f;

            if (animController_ && !idleAnimPath_.empty())
            {
                AnimationParameters params(context_, idleAnimPath_);
                params.looped_ = true;
                animController_->PlayNewExclusive(params);
                animController_->SetTime(idleAnimPath_, 0.0f);
                animController_->SetSpeed(idleAnimPath_, 1.0f);
            }
        }
    }

    // Lock fighter position - physics should not drift, but enforce it
    if (rigidBody_ && !isSwinging_)
    {
        Vector3 pos = rigidBody_->GetPosition();
        Vector3 drift = arenaPosition_ - pos;
        drift.y_ = 0.0f;
        if (drift.Length() > 0.001f)
        {
            rigidBody_->SetPosition(arenaPosition_);
            rigidBody_->SetLinearVelocity(Vector3::ZERO);
        }
    }
}

void Fighter::ReparentWeapon(Node* meshNode)
{
    if (!meshNode) return;

    auto* animatedModel = meshNode->FindComponent<AnimatedModel>();
    if (!animatedModel) return;
}

```

```

Skeleton& skeleton = animatedModel->GetSkeleton();
Bone* rightHandBone = skeleton.GetBone("RightHand");

if (rightHandBone && rightHandBone->node_)
{
    ea::vector<Node*> allDescendants;
    ea::vector<Node*> stack;
    stack.push_back(meshNode);

    while (!stack.empty())
    {
        Node* current = stack.back();
        stack.pop_back();
        for (Node* child : current->GetChildren())
        {
            allDescendants.push_back(child);
            stack.push_back(child);
        }
    }

    for (Node* child : allDescendants)
    {
        auto* sm = child->GetComponent<StaticModel>();
        if (!sm) continue;
        Model* mdl = sm->GetModel();
        if (!mdl) continue;

        const char* mdlName = mdl->GetName().c_str();
        if (std::strstr(mdlName, "BattleAxe") == nullptr) continue;

        Vector3 worldPos = child->GetWorldPosition();
        Quaternion worldRot = child->GetWorldRotation();

        child->SetParent(rightHandBone->node_);
        Quaternion handInv = rightHandBone->node_->GetWorldRotation().Inverse();
        child->SetPosition(handInv * (worldPos - rightHandBone->node_->GetWorldPosition()));
        child->SetRotation(handInv * worldRot);
    }
}
}
"Fighter.h"
-----
#pragma once

#include <Urho3D/Scene/LogicComponent.h>
#include <Urho3D/Graphics/AnimationController.h>
#include <Urho3D/Physics/RigidBody.h>
#include <Urho3D/Physics/CollisionShape.h>
#include <Urho3D/Graphics/AnimatedModel.h>
#include <Urho3D/Graphics/Skeleton.h>

using namespace Urho3D;

class Fighter : public LogicComponent
{
    URHO3D_OBJECT(Fighter, LogicComponent);

public:
    Fighter(Context* context);
    ~Fighter() override = default;

    static void RegisterObject(Context* context);

    void Initialize(Node* node, const ea::string& prefabPath);
    void Initialize(Node* node, const ea::string& idleAnimPath, const ea::string& attackAnimPath);

    void Reset();
    void StartAttack();

```



```

    void FixedUpdate(float timeStep) override;

    bool IsSwinging() const { return isSwinging_; }
    bool IsMidSwing() const { return isSwinging_ && (swingTimer_ >= 0.20f && swingTimer_ <=
0.60f); }
    Vector3 GetPosition() const { return node_ ? node_->GetPosition() : Vector3::ZERO; }
    RigidBody* GetRigidBody() const { return rigidBody_; }

private:
    void ReparentWeapon(Node* meshNode);
    void CancelRootMotion();
    void SetupPhysics();

    SharedPtr<Node> node_;
    SharedPtr<Node> meshNode_;
    SharedPtr<AnimationController> animController_;
    SharedPtr<RigidBody> rigidBody_;
    SharedPtr<CollisionShape> collisionShape_;
    AnimatedModel* animatedModel_ = nullptr;
    Bone* rootBone_ = nullptr;
    bool rootPosInitialized_ = false;
    float baselineY_ = 0.0f;

    ea::string idleAnimPath_;
    ea::string attackAnimPath_;

    bool isSwinging_{false};
    float swingTimer_{0.0f};
    const float maxSwingDuration_{1.2f};
    Vector3 arenaPosition_{Vector3::ZERO};
};

```

"main.cpp"

```

#include <Urho3D/Urho3D.h>
#include <Urho3D/Engine/Engine.h>
#include <Urho3D/Engine/EngineDefs.h>
#include <Urho3D/Graphics/Camera.h>
#include <Urho3D/Graphics/Light.h>
#include <Urho3D/Graphics/Octree.h>
#include <Urho3D/Graphics/Renderer.h>
#include <Urho3D/Graphics/Viewport.h>
#include <Urho3D/Graphics/Zone.h>
#include <Urho3D/Graphics/DebugRenderer.h>
#include <Urho3D/Graphics/StaticModel.h>
#include <Urho3D/Graphics/Model.h>
#include <Urho3D/Graphics/Material.h>
#include <Urho3D/Container/Str.h>
#include <Urho3D/IO/Log.h>
#include <Urho3D/Resource/ResourceCache.h>
#include <Urho3D/Scene/Scene.h>
#include <Urho3D/Scene/PrefabResource.h>
#include <Urho3D/Input/Input.h>
#include <Urho3D/Graphics/GraphicsDefs.h>
#include <Urho3D/UI/Text.h>
#include <Urho3D/UI/UI.h>
#include <Urho3D/UI/Font.h>
#include <Urho3D/Core/Timer.h>
#include <Urho3D/Physics/PhysicsWorld.h>

// Include our new object-oriented components
#include "Fighter.h"
#include "Zombie.h"
#include "CombatSystem.h"

```

```
using namespace Urho3D;
```

```
int main(int argc, char** argv)
{

```

```
SharedPtr<Context> context(new Context());
SharedPtr<Engine> engine(new Engine(context));

StringVariantMap engineParameters;
engineParameters[EP_WINDOW_TITLE] = "Kilo-Code Battle Arena";
engineParameters[EP_RESOURCE_PREFIX_PATHS] = "Data";
engineParameters[EP_FULL_SCREEN] = false;
engineParameters[EP_WINDOW_WIDTH] = 1152;
engineParameters[EP_WINDOW_HEIGHT] = 864;
engineParameters[EP_MULTI_SAMPLE] = 1;
engineParameters[EP_VSYNC] = true;

if (!engine->Initialize(engineParameters, {}))
{
    URH3D_LOGERROR("Could not initialise the engine.");
    return 1;
}

// REGISTER CUSTOM COMPONENT FACTORIES SO THE ENGINE CAN REFLECT THEM
context->RegisterFactory<Fighter>();
context->RegisterFactory<Zombie>();
context->RegisterFactory<CombatSystem>();

auto* renderer = engine->GetSubsystem<Renderer>();
auto* cache = engine->GetSubsystem<ResourceCache>();
auto* input = engine->GetSubsystem<Input>();
auto* ui = engine->GetSubsystem<UI>();
auto* time = engine->GetSubsystem<Time>();

// ---- Diagnostics HUD Overlay ----
auto* font = cache->GetResource<Font>("Fonts/Anonymous Pro.ttf");
auto* debugText = ui->GetRoot()->CreateChild<Text>();
if (font && debugText)
{
    debugText->SetText("SPACEBAR: ATTACK / RESPAWN | W/S: ZOOM | A/D: ORBIT");
    debugText->SetFont(font, 16);
    debugText->SetColor(Color::GREEN);
    debugText->SetHorizontalAlignment(HA_CENTER);
    debugText->SetVerticalAlignment(VA_TOP);
}

// ---- World Scene Layout Initialization ----
SharedPtr<Scene> scene(new Scene(context));
scene->CreateComponent<Octree>();
scene->CreateComponent<DebugRenderer>();
scene->CreateComponent<PhysicsWorld>(); // Required for RigidBody components

auto* z = scene->CreateComponent<Zone>();
z->SetBoundingBox(BoundingBox(-1000.0f, 1000.0f));
z->SetAmbientColor(Color(0.35f, 0.35f, 0.35f));
z->SetFogColor(Color(0.55f, 0.55f, 0.55f));
z->SetFogStart(150.0f);
z->SetFogEnd(250.0f);

Node* l = scene->CreateChild("KeyLight");
l->SetDirection(Vector3(0.2f, -0.8f, 0.5f));
auto* dl = l->CreateComponent<Light>();
dl->SetLightType(LIGHT_DIRECTIONAL);
dl->SetColor(Color(0.8f, 0.784f, 0.76f));
dl->SetBrightness(0.72f);
dl->SetCastShadows(false);

Node* fillLightNode = scene->CreateChild("FillLight");
fillLightNode->SetDirection(Vector3(-0.4f, -0.6f, -0.2f));
auto* fillLight = fillLightNode->CreateComponent<Light>();
fillLight->SetLightType(LIGHT_DIRECTIONAL);
fillLight->SetColor(Color(0.4f, 0.4f, 0.44f));
fillLight->SetBrightness(0.32f);
```

```

auto* cameraNode = scene->CreateChild("Camera");
cameraNode->SetPosition(Vector3(0.0f, 1.2f, -3.5f));
cameraNode->LookAt(Vector3(0.0f, 0.6f, 1.6f));
auto* camera = cameraNode->CreateComponent<Camera>();

Node* camLightNode = scene->CreateChild("ViewerLight");
camLightNode->SetPosition(cameraNode->GetPosition());
camLightNode->SetRotation(cameraNode->GetRotation());
auto* camLight = camLightNode->CreateComponent<Light>();
camLight->SetLightType(LIGHT_SPOT);
camLight->SetColor(Color(1.2f, 1.12f, 1.04f));
camLight->SetBrightness(3.2f);
camLight->SetRange(40.0f);
camLight->SetFov(72.0f);
camLight->SetCastShadows(false);

SharedPtr<Viewport> viewport(new Viewport(context, scene, camera));
viewport->SetDrawDebug(true);
renderer->SetViewport(0, viewport);

// =====
// COMPONENT CREATION & BINDING POOL
// =====

// 1. Create and attach the Fighter component
Node* axeManPivot = scene->CreateChild("AxeManPivot");
axeManPivot->SetPosition(Vector3(0.59f, 0.0f, 1.6f));
auto* fighterComponent = axeManPivot->CreateComponent<Fighter>();
fighterComponent->Initialize(axeManPivot, "test.glb.d/Prefab.prefab");

// 2. Create and attach the Zombie component
Node* zombiePivot = scene->CreateChild("ZombiePivot");
zombiePivot->SetPosition(Vector3(-2.0f, 0.0f, 1.6f)); // Moved closer to be visible on screen
auto* zombieComponent = zombiePivot->CreateComponent<Zombie>();
const ea::string zombiePrefabPath = "firing-running-
didn'twork/test_out/test.glTF.d/Prefab.prefab";
zombieComponent->Initialize(zombiePivot, zombiePrefabPath);

// 3. Setup the Master Combat System Arbitrator Component
Node* coreSystemNode = scene->CreateChild("CombatSystemNode");
auto* combatSystem = coreSystemNode->CreateComponent<CombatSystem>();
combatSystem->Initialize(fighterComponent, zombieComponent, debugText);

// =====
// MAIN LOOP
// =====
while (!engine->IsExiting())
{
    engine->RunFrame();
    float dt = time->GetTimeStep();

    // --- ZOMBIE AUTOPSY: run every frame, but only log every 30th to save IO ---
    static int debugFrame = 0;
    debugFrame++;
    bool logNow = (debugFrame % 30 == 1);

    if (logNow && zombieComponent)
    {
        Node* zPivot = zombieComponent->GetNode();
        if (zPivot)
        {
            URH3D_LOGINFOF("[Z-AUTOPSY] Pivot    pos=%s rot=%s",
                          zPivot->GetPosition().ToString().c_str(),
                          zPivot->GetRotation().ToString().c_str());

            Node* zMesh = zPivot->GetChild("ZombieMesh");
            if (zMesh)
            {
                URH3D_LOGINFOF("[Z-AUTOPSY] Mesh      worldPos=%s scale=%s",

```

```

zMesh->GetWorldPosition().ToString().c_str(),
zMesh->GetScale().ToString().c_str());

AnimatedModel* am = zMesh->FindComponent<AnimatedModel>();
if (!am) am = zPivot->FindComponent<AnimatedModel>(); // fallback
if (am)
{
    BoundingBox bb = am->GetWorldBoundingBox();
    URH3D_LOGINFOF("[Z-AUTOPSY] WorldBB min=%s max=%s size=%s",
        bb.min_.ToString().c_str(),
        bb.max_.ToString().c_str(),
        bb.Size().ToString().c_str());
    URH3D_LOGINFOF("[Z-AUTOPSY] Drawable enabled=%d castShadows=%d",
        am->IsEnabled(), am->GetCastShadows());
}
else
{
    URH3D_LOGWARNING("[Z-AUTOPSY] NO AnimatedModel found!");
}
}
else
{
    URH3D_LOGWARNING("[Z-AUTOPSY] NO 'ZombieMesh' child found!");
}

RigidBody* zBody = zombieComponent->GetRigidBody();
if (zBody)
{
    URH3D_LOGINFOF("[Z-AUTOPSY] Physics pos=%s vel=%s mass=%.1f",
        zBody->GetPosition().ToString().c_str(),
        zBody->GetLinearVelocity().ToString().c_str(),
        zBody->GetMass());
}
}

// --- VISUAL DEBUG DRAW ---
auto* dbg = scene->GetComponent<DebugRenderer>();
if (dbg)
{
    dbg->SetEnabled(true); // make sure it's on

    if (zombieComponent && zombieComponent->GetNode())
    {
        Node* zPivot = zombieComponent->GetNode();
        // Red wire sphere at pivot
        dbg->AddSphere(Sphere(zPivot->GetWorldPosition(), 0.2f), Color::RED);

        Node* zMesh = zPivot->GetChild("ZombieMesh");
        if (zMesh)
        {
            AnimatedModel* am = zMesh->FindComponent<AnimatedModel>();
            if (am)
            {
                // Green wire bounding box of the actual model
                dbg->AddBoundingBox(am->GetWorldBoundingBox(), Color::GREEN);
            }
        }
    }

    // Also draw fighter for reference
    if (fighterComponent && fighterComponent->GetNode())
    {
        Node* fPivot = fighterComponent->GetNode();
        dbg->AddSphere(Sphere(fPivot->GetWorldPosition(), 0.2f), Color::BLUE);
        AnimatedModel* fam = fPivot->FindComponent<AnimatedModel>();
        if (!fam && fPivot->GetChild("AxeManMesh"))
            fam = fPivot->GetChild("AxeManMesh")->FindComponent<AnimatedModel>();
        if (fam)
    }
}

```

```

        dbg->AddBoundingBox(fam->GetWorldBoundingBox(), Color::CYAN);
    }
}

if (input->GetKeyPress(KEY_SPACE))
{
    if (combatSystem->IsArenaHitFrozen())
    {
        combatSystem->ResetSimulation();
    }
    else
    {
        combatSystem->HandleManualAttack();
    }
}

combatSystem->UpdateCombat(dt);

// Camera controls
const Vector3 screenCenterAnchor(0.0f, 0.6f, 1.6f);
Vector3 anchorToCamVector = cameraNode->GetPosition() - screenCenterAnchor;

bool orbitLeft  = input->GetKeyDown(KEY_A) || input->GetKeyDown(KEY_Q);
bool orbitRight = input->GetKeyDown(KEY_D) || input->GetKeyDown(KEY_E);

if (orbitLeft || orbitRight)
{
    float rotationDirection = orbitLeft ? -90.0f : 90.0f;
    rotationDirection *= dt;
    Quaternion yawOrbitRotation(rotationDirection, Vector3::UP);
    anchorToCamVector = yawOrbitRotation * anchorToCamVector;
    cameraNode->SetPosition(screenCenterAnchor + anchorToCamVector);
}

if (input->GetKeyDown(KEY_W)) cameraNode->Translate(Vector3::FORWARD * 3.0f * dt);
if (input->GetKeyDown(KEY_S)) cameraNode->Translate(Vector3::BACK * 3.0f * dt);

cameraNode->LookAt(screenCenterAnchor);

camLightNode->SetPosition(cameraNode->GetPosition());
camLightNode->SetRotation(cameraNode->GetRotation());

if (input->GetKeyPress(KEY_ESCAPE)) engine->Exit();

}

renderer->SetViewport(0, nullptr);
engine->Exit();

return 0;
}
"Zombie.cpp"
-----
#include "Zombie.h"
#include "Fighter.h"
#include <Urho3D/IO/Log.h>
#include <Urho3D/Core/Context.h>
#include <Urho3D/Resource/ResourceCache.h>
#include <Urho3D/Scene/PrefabResource.h>
#include <Urho3D/Physics/PhysicsWorld.h>
#include <cmath>

Zombie::Zombie(Context* context) :
    LogicComponent(context)
{
    SetUpdateEventMask(USE_FIXEDUPDATE);
}

void Zombie::RegisterObject(Context* context)

```

```
{
    context->RegisterFactory<Zombie>();
}

static AnimatedModel* FindAnimatedModelRecursive(Node* node)
{
    if (!node) return nullptr;
    AnimatedModel* am = node->GetComponent<AnimatedModel>();
    if (am) return am;
    for (Node* child : node->GetChildren())
    {
        am = FindAnimatedModelRecursive(child);
        if (am) return am;
    }
    return nullptr;
}

void Zombie::Initialize(Node* node, const ea::string& prefabPath)
{
    node_ = node;
    if (!node_) return;

    startPosition_ = node_->GetPosition();

    node_->SetPosition(startPosition_);
    node_->SetRotation(Quaternion(0.0f, 90.0f, 0.0f));

    size_t lastSlash = prefabPath.find_last_of('/');
    if (lastSlash != ea::string::npos)
    {
        ea::string baseDir = prefabPath.substr(0, lastSlash);
        runAnimPath_ = baseDir + "/Animations/mixamo.com.ani";
    }
    else
    {
        runAnimPath_ = prefabPath;
    }
    URH03D_LOGINFOF("[ZOMBIE-INIT] Animation path derived: %s", runAnimPath_.c_str());

    meshNode_ = node_->CreateChild("ZombieMesh");
    auto* cache = GetSubsystem<ResourceCache>();
    auto* prefab = cache->GetResource<PrefabResource>(prefabPath);

    if (prefab)
    {
        prefab->InstantiateReference(meshNode_);
        animController_ = meshNode_->FindComponent<AnimationController>();

        meshNode_->SetScale(75.0f);
        meshNode_->SetPosition(Vector3::ZERO);
        meshNode_->SetRotation(Quaternion::IDENTITY);

        animatedModel_ = meshNode_->FindComponent<AnimatedModel>();
        if (!animatedModel_)
            animatedModel_ = FindAnimatedModelRecursive(meshNode_);

        if (animatedModel_)
        {
            Skeleton& skeleton = animatedModel_->GetSkeleton();
            const char* rootNames[] = {
                "mixamorig:Hips", "mixamorig_Hips", "Hips",
                "mixamorig:Root", "mixamorig_Root", "Root",
                "mixamorig:Pelvis", "mixamorig_Pelvis", "Pelvis"
            };
            for (const char* name : rootNames)
            {
                rootBone_ = skeleton.GetBone(name);
                if (rootBone_) break;
            }
        }
    }
}
```

```

        if (rootBone_)
            URHO3D_LOGINFOF("[ZOMBIE-INIT] Root bone found: %s", rootBone_>name_.c_str());
        else
            URHO3D_LOGWARNING("[ZOMBIE-INIT] Root bone NOT found! Treadmill will not work.");
    }

    SetupPhysics();
    Reset();
}
else
{
    URHO3D_LOGERRORF("[ZOMBIE-INIT] Failed to load prefab: %s", prefabPath.c_str());
}
}

void Zombie::SetupPhysics()
{
    if (!node_) return;

    rigidBody_ = node_>GetOrCreateComponent<RigidBody>();
    if (rigidBody_)
    {
        rigidBody_>SetMass(60.0f);
        rigidBody_>SetUseGravity(false);
        rigidBody_>SetLinearDamping(5.0f); // Less damping than fighter, can move
        rigidBody_>SetAngularDamping(10.0f);
        rigidBody_>SetKinematic(false);
        rigidBody_>SetCollisionLayer(4); // Zombie layer
        rigidBody_>SetCollisionMask(0xFFFFFFFF);

        URHO3D_LOGINFO("[ZOMBIE-PHYSICS] RigidBody created. Mass=60kg, Gravity=OFF,
Kinematic=OFF");
    }

    collisionShape_ = node_>GetOrCreateComponent<CollisionShape>();
    if (collisionShape_)
    {
        collisionShape_>SetCapsule(0.3f, 1.6f, Vector3(0.0f, 0.8f, 0.0f));
        URHO3D_LOGINFO("[ZOMBIE-PHYSICS] Capsule collider created. R=0.3, H=1.6");
    }
}

void Zombie::FixedUpdate(float timeStep)
{
    if (node_)
        URHO3D_LOGINFOF("[ZOMBIE-POS] x=%.2f", node_>GetPosition().x_);
    if (health_ <= 0) return;
    if (isFrozen_) return;

    if (isNeckBiting_)
    {
        biteTimer_ += timeStep;
        if (biteTimer_ >= biteDuration_)
        {
            isNeckBiting_ = false;
            isContactFrozen_ = false;
            biteTimer_ = 0.0f;
            URHO3D_LOGINFO("[ZOMBIE-BITE] Neck bite complete. Resuming approach.");
        }

        // During bite, zero velocity
        if (rigidBody_)
        {
            rigidBody_>SetLinearVelocity(Vector3::ZERO);
        }

        if (animController_ && animController_>IsPlaying(runAnimPath_))
            CancelRootMotion();
    }
}

```

```

        return;
    }

    // === DIRECT POSITION DRIVE - bypass Bullet entirely ===
    if (!isContactFrozen_)
    {
        // Move the pivot manually
        Vector3 pos = node_->GetPosition();
        pos.x_ += runSpeed_ * timeStep;
        node_->SetPosition(pos);

        // Snap physics body to follow the node (collision detection only)
        if (rigidBody_)
        {
            rigidBody_->SetPosition(pos);
            rigidBody_->SetLinearVelocity(Vector3::ZERO);
        }

        URH3D_LOGINFOF("[ZOMBIE-MOVE] Manual move to x=%.3f", pos.x_);
    }
    else if (isContactFrozen_ && rigidBody_)
    {
        rigidBody_->SetLinearVelocity(Vector3::ZERO);
    }

    if (animController_ && animController_->IsPlaying(runAnimPath_))
        CancelRootMotion();
}

void Zombie::CancelRootMotion()
{
    if (!rootBone_ || !rootBone_->node_ || !meshNode_ || !node_) return;

    Vector3 rootWorld = rootBone_->node_->GetWorldPosition();
    Vector3 nodeWorld = node_->GetWorldPosition();

    if (!rootPosInitialized_)
    {
        baselineY_ = rootWorld.y_ - nodeWorld.y_;
        rootPosInitialized_ = true;
    }

    Vector3 desired(nodeWorld.x_, nodeWorld.y_ + baselineY_, nodeWorld.z_);
    Vector3 delta = desired - rootWorld;

    if (delta.Length() > 0.001f)
    {
        meshNode_->Translate(delta, TS_WORLD);
    }
}

void Zombie::CheckCollision(Fighter* fighter)
{
    if (!node_ || !fighter || isFrozen_ || health_ <= 0 || isNeckBiting_) return;

    // PHYSICS-DRIVEN: Use actual positions from physics bodies
    Vector3 myPos = node_->GetPosition();
    Vector3 targetPos = fighter->GetPosition();
    float boundaryX = targetPos.x_ - solidStopThreshold_;

    // DEBUG: log positions every frame when within range
    float deltaX = myPos.x_ - targetPos.x_;
    if (std::abs(deltaX) < 3.0f)
    {
        URH3D_LOGINFOF("[ZOMBIE-COLLISION] MyX=%.3f | TargetX=%.3f | BoundaryX=%.3f | Delta=%.3f  

| Frozen=%d",
                        myPos.x_, targetPos.x_, boundaryX, deltaX, isContactFrozen_ ? 1 : 0);
    }
}

```



```

// Already at contact - maintain exact position
if (isContactFrozen_)
{
    // Physics should prevent merging, but add safety check
    if (myPos.x_ > boundaryX)
    {
        node_->SetPosition(Vector3(boundaryX, myPos.y_, myPos.z_));
        if (rigidBody_)
        {
            rigidBody_->SetPosition(Vector3(boundaryX, myPos.y_, myPos.z_));
            rigidBody_->SetLinearVelocity(Vector3::ZERO);
        }
        URH03D_LOGINFOF("[ZOMBIE-COLLISION] PUSHBACK: Hero at %.3f, zombie pushed to %.3f",
targetPos.x_, boundaryX);
    }
    return;
}

// Zombie reached axe man body - stop at contact line
if (myPos.x_ >= boundaryX)
{
    node_->SetPosition(Vector3(boundaryX, myPos.y_, myPos.z_));
    if (rigidBody_)
    {
        rigidBody_->SetPosition(Vector3(boundaryX, myPos.y_, myPos.z_));
        rigidBody_->SetLinearVelocity(Vector3::ZERO);
    }
    isContactFrozen_ = true;
    URH03D_LOGINFOF("[ZOMBIE-COLLISION] CONTACT FREEZE at boundaryX=%.3f | Gap=%.3f",
boundaryX, targetPos.x_ - boundaryX);
}

// Zombie passed through without contact - reset
if (myPos.x_ >= targetPos.x_ + solidStopThreshold_)
{
    URH03D_LOGINFOF("[ZOMBIE-COLLISION] PASSED THROUGH fighter! MyX=%.3f TargetX=%.3f.
Resetting.", myPos.x_, targetPos.x_);
    Reset();
}
}

void Zombie::HandleHit(Fighter* fighter)
{
    if (hasBeenHit_) return;
    hasBeenHit_ = true;
    isFrozen_ = true;
    isNeckBiting_ = false;

    URH03D_LOGINFO("[ZOMBIE-HIT] AXE COLLISION REGISTERED. Freezing.");

    if (animController_)
    {
        animController_->StopAll();
    }

    if (rigidBody_)
    {
        rigidBody_->SetLinearVelocity(Vector3::ZERO);
        rigidBody_->SetAngularVelocity(Vector3::ZERO);
    }
}

void Zombie::ApplyBoundaryFreeze()
{
    if (isContactFrozen_) return;

    isContactFrozen_ = true;
    URH03D_LOGINFO("[ZOMBIE-LOCK] Gate activated. Absolute freeze applied.");
}

```

```
    if (rigidBody_)
    {
        rigidBody_>SetLinearVelocity(Vector3::ZERO);
    }
}

void Zombie::TriggerNeckBite()
{
    if (isNeckBiting_) return;

    isNeckBiting_ = true;
    biteTimer_ = 0.0f;
    URH03D_LOGINFO("[ZOMBIE-BITE] Starting neck bite sequence!");

    if (rigidBody_)
    {
        rigidBody_>SetLinearVelocity(Vector3::ZERO);
    }
}

void Zombie::StartNeckBite(Node* targetNode)
{
    TriggerNeckBite();
}

void Zombie::Reset()
{
    isFrozen_ = false;
    isContactFrozen_ = false;
    isNeckBiting_ = false;
    hasBeenHit_ = false;
    health_ = maxHealth_;
    rootPosInitialized_ = false;
    biteTimer_ = 0.0f;

    if (node_)
    {
        node_>SetPosition(startPosition_);
        node_>SetRotation(Quaternion(0.0f, 90.0f, 0.0f));

        if (rigidBody_)
        {
            rigidBody_>SetPosition(startPosition_);
            rigidBody_>SetRotation(Quaternion(0.0f, 90.0f, 0.0f));
            rigidBody_>SetLinearVelocity(Vector3::ZERO);
            rigidBody_>SetAngularVelocity(Vector3::ZERO);
            rigidBody_>ResetForces();
            rigidBody_>Activate();
        }
    }

    if (meshNode_)
        meshNode_>SetPosition(Vector3::ZERO);

    if (animController_ && !runAnimPath_.empty())
    {
        animController_>StopAll();

        AnimationParameters params(context_, runAnimPath_);
        params.looped_ = true;
        animController_>PlayNewExclusive(params);
        animController_>SetTime(runAnimPath_, 0.0f);
        animController_>SetSpeed(runAnimPath_, 0.5f);

        URH03D_LOGINFOF("[ZOMBIE-RESET] Run animation started at (%.2f, %.2f, %.2f)",
            startPosition_.x_, startPosition_.y_, startPosition_.z_);
    }
    else
    {

```

```

        URHO3D_LOGWARNING("[ZOMBIE-RESET] Cannot start run animation – controller or path
missing!");
    }
}
"Zombie.h"
-----
#pragma once

#include <Urho3D/Scene/LogicComponent.h>
#include <Urho3D/Graphics/AnimationController.h>
#include <Urho3D/Physics/RigidBody.h>
#include <Urho3D/Physics/CollisionShape.h>
#include <Urho3D/Graphics/AnimatedModel.h>
#include <Urho3D/Graphics/Skeleton.h>

using namespace Urho3D;

// FORWARD DECLARATION
class Fighter;

class Zombie : public LogicComponent
{
    URHO3D_OBJECT(Zombie, LogicComponent);

public:
    Zombie(Context* context);
    ~Zombie() override = default;

    static void RegisterObject(Context* context);

    void Initialize(Node* node, const ea::string& prefabPath);
    void Reset();

    void FixedUpdate(float timeStep) override;

    void TriggerNeckBite();
    void StartNeckBite(Node* targetNode);
    void ApplyBoundaryFreeze();
    void CheckCollision(Fighter* fighter);
    void HandleHit(Fighter* fighter);

    bool IsFrozen() const { return isFrozen_; }
    bool IsContactFrozen() const { return isContactFrozen_; }
    bool IsNeckBiting() const { return isNeckBiting_; }
    Vector3 GetPosition() const { return node_ ? node_->GetPosition() : Vector3::ZERO; }
    RigidBody* GetRigidBody() const { return rigidBody_; }

private:
    void CancelRootMotion();
    void SetupPhysics();

    SharedPtr<Node> node_;
    SharedPtr<Node> meshNode_;
    SharedPtr<AnimationController> animController_;
    SharedPtr<RigidBody> rigidBody_;
    SharedPtr<CollisionShape> collisionShape_;
    AnimatedModel* animatedModel_ = nullptr;
    Bone* rootBone_ = nullptr;
    bool rootPosInitialized_ = false;
    float baselineY_ = 0.0f;

    ea::string runAnimPath_;
    Vector3 startPosition_{-4.5f, 0.0f, 1.6f};

    bool isFrozen_{false};
    bool isContactFrozen_{false};
    bool isNeckBiting_{false};

    float health_{100.0f};

```

```
const float maxHealth_{100.0f};  
const float runSpeed_{0.90f};  
const float solidStopThreshold_{1.2f};  
bool hasBeenHit_ = false;  
  
float biteTimer_ = 0.0f;  
const float biteDuration_ = 2.5f;  
};
```